CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 7

PHP Form Processing with Server-Side Validation and JSON File Storage

Student Name	Shreyas
Roll Number	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

1. Aim and Objectives

To process a StudentHub inquiry (contact) form with PHP: receive the data with the POST method, validate and sanitise every field on the server, store valid records in a JSON file safely, and show clear success and error messages.

- Build an accessible PHP form whose data is sent with POST and processed only on the server.
- Validate empty and invalid inputs on the server, including the email address, without relying on JavaScript or browser checks.
- Sanitise the input and escape every value when it is displayed, so submitted text can never run as HTML.
- Store valid submissions in a JSON file with file locking, and handle a missing or damaged file.
- Show a clear success message after saving and clear messages next to each invalid field.
- Intermediate extension: display the stored JSON records on a web page.
- Advanced extension: protect the form with a CSRF token.

2. Problem Definition

Official problem definition: *"Process submitted registration/contact form data using PHP. Validate and sanitize inputs on the server side and store records in CSV or JSON file format. Display success/error messages."*

The Practical 7 folder contained a static contact.html page that opened blank (it waited for missing login scripts) and posted to api/contact.php, which stored data in CSV. Its form was pre-filled with personal details, error messages were passed back through the page address, and HTML escaping was applied before storing, which changes the stored text. This practical keeps the same idea – a student inquiry form with server-side validation, sanitisation and file locking – and rebuilds it in PHP with JSON storage, as required.

The practical list asks four key questions; the answers for this implementation are:

Key question	Answer in this practical
Is the form submitted using POST?	Yes. The form has method="post"; index.php processes data only when $_SERVER['REQUEST_METHOD'] is POST. A GET request with the same data in the address stores nothing (tested).
Are inputs validated and sanitized server side?	Yes. PHP checks required fields, name pattern and length, email (filter_var), category list and message length, and removes control characters and extra spaces. The browser's own checks are switched off (novalidate) to show this.
Is file writing handled safely?	The JSON file is read, changed and written under an exclusive flock() lock; 10 simultaneous submissions were all stored. A missing file or folder is created; a damaged file is backed up and replaced. The file is not served to browsers.
Are success and error responses displayed clearly?	A success message with a reference number after a redirect; on errors, a summary of the fields to correct and a message under each field, with the entered values kept.

3. Tools and Technologies

Tool / technology	Use in this practical
PHP 8.2.12 (XAMPP)	Form processing, validation, sanitisation, sessions, JSON storage; built-in web server
XAMPP	Provides PHP on Windows (C:\xampp\php\php.exe); Apache is an alternative way to run the project
HTML5, CSS3	Form markup with labels and ARIA attributes; responsive layout with system fonts
JSON	Storage format of the submissions (private/submissions.json)
Python 3.12.0	Automated HTTP tests (tests/test_form.py) and the screenshot run
Microsoft Edge 156.0.4314.50	Viewing the pages; headless mode for the screenshots
Visual Studio Code, Git	Source code editor; version control of the StudentHub repository
Operating system	Windows 11

4. Project Structure and How to Run

```
Practical-7/
├── index.php                form and POST processing
├── records.php              stored submissions (table + JSON)
├── includes/functions.php   escaping, sanitising, validation, CSRF
├── includes/storage.php     JSON read/append with flock()
├── assets/css/style.css
├── router.php               blocks private/ and includes/ on PHP's built-in server
├── start-server.bat         starts C:\xampp\php\php.exe -S 127.0.0.1:8607 router.php
├── private/                 submissions.json (not committed) + .htaccess
├── sample-output/submissions.sample.json
├── screenshots/
└── tests/test_form.py
```

- **Option A (tested):** double-click start-server.bat, or run C:\xampp\php\php.exe -S 127.0.0.1:8607 router.php in the Practical-7 folder, then open http://127.0.0.1:8607/index.php.
- **Option B (not tested):** copy the folder to C:\xampp\htdocs\practical7, start Apache in the XAMPP Control Panel and open http://localhost/practical7/index.php. The .htaccess files in private/ and includes/ deny direct access.
- **Port 8607:** on the development PC, port 8000 was already used by another PHP server and port 80 by XAMPP Apache, so a separate port and the 127.0.0.1 address are used.
- **Another computer:** copy the Practical-7 folder (relative paths only) and install XAMPP; the JSON file is created automatically.

5. Form and POST Processing

index.php shows the form and also processes it. The form posts to itself; the hidden csrf_token field carries the CSRF token. When the request is a POST, the data is sanitised, the token is checked, the fields are validated, and a valid record is appended to the JSON file. After saving, the page redirects with HTTP 303

(Post/Redirect/Get), so refreshing the success page cannot submit the form again. Invalid data returns HTTP 422 with the form shown again; a missing or wrong token returns HTTP 400.

5.1 index.php – the form element

```
<form action="index.php" method="post" novalidate>
    <input type="hidden" name="csrf_token" value="<?= e($token) ?>">
```

5.2 index.php – POST processing

```
$values = ['name' => '', 'email' => '', 'category' => '', 'message' => ''];
$errors = [];
$formError = '';
$success = $_SESSION['flash'] ?? null;                // set after a successful POST, shown once
unset($_SESSION['flash']);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $values = sanitize_submission($_POST);

    if (!csrf_is_valid($_POST['csrf_token'] ?? null)) {
        http_response_code(400);
        $formError = 'Your form session has expired or the request was not sent from this form. Please check your details and submit again.';
    } else {
        $errors = validate_submission($values, $_POST);
        if ($errors) {
            http_response_code(422);
        } else {
            try {
                $saved = append_submission([
                    'submitted_at' => date('c'),
                    'name'         => $values['name'],
                    'email'        => $values['email'],
                    'category'     => $values['category'],
                    'message'      => $values['message'],
                ]);
                $_SESSION['flash'] = ['id' => $saved['id'], 'name' => $saved['name']];
                unset($_SESSION['csrf_token']);         // a new token for the next form
                // Post/Redirect/Get: refreshing the success page cannot submit the form twice
                header('Location: index.php?status=sent', true, 303);
                exit;
            } catch (RuntimeException | JsonException $exception) {
                error_log('Practical 7 storage error: ' . $exception->getMessage());
                http_response_code(500);
                $formError = 'Sorry, your inquiry could not be saved because of a server problem. Please try again later.';
            }
        }
    }
```

6. Server-Side Validation

Field	Rule	Message
Full name	Required; 2–60 characters; letters, spaces, dots, apostrophes, hyphens (/^\p{L}[\p{L} .'-]*$/u)	Your name can contain only letters, spaces, dots, apostrophes and hyphens.
Email	Required; at most 100 characters; filter_var($email, FILTER_VALIDATE_EMAIL)	Please enter a valid email address, for example name@example.com.
Category	Required; must be one of the five keys in CATEGORIES (a tampered value is rejected)	Please choose a category from the list.
Message	Required; 10–1000 characters	Your message must be at least 10 characters long.
All fields	Must be valid UTF-8 text (arrays such as name[] count as empty)	This field contains characters that are not allowed.

The HTML form also has required, maxlength and type="email" for accessibility, but it carries the novalidate attribute so that the browser does not block the request: every check shown in this report is performed by PHP. The automated tests send requests directly to the server, bypassing the browser completely.

6.1 functions.php – validate_submission()

```php
// Server-side validation. Returns field => message for every problem (empty array = valid).
function validate_submission(array $data, array $raw): array
{
    $errors = [];
    foreach (['name', 'email', 'category', 'message'] as $field) {
        if (!mb_check_encoding(post_string($raw, $field), 'UTF-8')) {
            $errors[$field] = 'This field contains characters that are not allowed.';
        }
    }

    if (!isset($errors['name'])) {
        $length = mb_strlen($data['name']);
        if ($data['name'] === '') {
            $errors['name'] = 'Please enter your full name.';
        } elseif ($length < 2 || $length > NAME_MAX) {
            $errors['name'] = 'Your name must be 2–' . NAME_MAX . ' characters long.';
        } elseif (!preg_match("/^\p{L}[\p{L} .'-]*$/u", $data['name'])) {
            $errors['name'] = 'Your name can contain only letters, spaces, dots, apostrophes and hyphens.';
        }
    }

    if (!isset($errors['email'])) {
        if ($data['email'] === '') {
            $errors['email'] = 'Please enter your email address.';
        } elseif (mb_strlen($data['email']) > EMAIL_MAX || filter_var($data['email'], FILTER_VALIDATE_EMAIL) === false) {
            $errors['email'] = 'Please enter a valid email address, for example name@example.com.';
        }
    }

    if (!isset($errors['category'])) {
        if ($data['category'] === '') {
            $errors['category'] = 'Please choose a category.';
        } elseif (!array_key_exists($data['category'], CATEGORIES)) {
            $errors['category'] = 'Please choose a category from the list.';
        }
    }

    if (!isset($errors['message'])) {
        $length = mb_strlen($data['message']);
        if ($data['message'] === '') {
            $errors['message'] = 'Please enter your message.';
        } elseif ($length < MESSAGE_MIN) {
            $errors['message'] = 'Your message must be at least ' . MESSAGE_MIN . ' characters long.';
        } elseif ($length > MESSAGE_MAX) {
            $errors['message'] = 'Your message must not be longer than ' . MESSAGE_MAX . ' characters.';
        }
    }
    return $errors;
}
```

7. Sanitisation and Safe Display

- **Text only:** post_string() accepts only strings, so an array sent as name[] becomes an empty value.
- **Cleaning:** clean_text() rejects invalid UTF-8, removes control characters (keeping new lines in the message), trims the value, tidies repeated spaces and caps the length; the email is converted to lower case.
- **Escape on output:** HTML in a submission is not trusted. Instead of changing the stored text, every value is escaped with e() (htmlspecialchars with ENT_QUOTES and UTF-8) at the moment it is printed. A message such as <script>alert("XSS")</script> is therefore displayed as text and never runs.

- **JSON file:** json_encode uses JSON_HEX_TAG, so < and > are stored as \u003C and \u003E; reading the file back gives the original text.

7.1 functions.php – escaping and sanitising

```
18  // Escape text for HTML output. Every value shown on a page goes through this function.
19  function e(string $value): string
20  {
21      return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
22  }
23
24  // Reads one POST field as a string; arrays or missing fields become ''.
25  function post_string(array $post, string $key): string
26  {
27      $value = $post[$key] ?? '';
28      return is_string($value) ? $value : '';
29  }
30
31  // Sanitising: valid UTF-8 only, no control characters, trimmed, whitespace tidied, length capped.
32  // The cleaned text is stored as plain text; HTML is never trusted, it is escaped when displayed.
33  function clean_text(string $value, int $maxLength, bool $multiline = false): string
34  {
35      if (!mb_check_encoding($value, 'UTF-8')) {
36          return '';
37      }
38      $value = str_replace(["\r\n", "\r"], "\n", $value);
39      $value = $multiline
40          ? preg_replace('/[^\P{C}\n\t]/u', '', $value)       // keep new lines and tabs in the message
41          : preg_replace('/\p{C}/u', ' ', $value);            // no control characters at all in one-line
    fields
42      $value = $multiline
43          ? preg_replace("/\n{3,}/", "\n\n", preg_replace('/[ \t]+/', ' ', $value))
44          : preg_replace('/\s+/u', ' ', $value);
45      return mb_substr(trim((string) $value), 0, $maxLength + 1);   // +1 so "too long" can still be
    reported
46  }
47
48  function sanitize_submission(array $post): array
49  {
50      return [
51          'name'     => clean_text(post_string($post, 'name'), NAME_MAX),
52          'email'    => mb_strtolower(clean_text(post_string($post, 'email'), EMAIL_MAX)),
53          'category' => clean_text(post_string($post, 'category'), 20),
54          'message'  => clean_text(post_string($post, 'message'), MESSAGE_MAX, true),
55      ];
56  }
```

7.2 records.php – escaped output

```
67  <td data-label="ID"><?= e($text($record['id'] ?? '')) ?></td>
68  <td data-label="Submitted"><?= e($formatTime($record['submitted_at'] ?? null)) ?></td>
69  <td data-label="Name"><?= e($text($record['name'] ?? '')) ?></td>
70  <td data-label="Email"><?= e($text($record['email'] ?? '')) ?></td>
71  <td data-label="Category"><?= e(CATEGORIES[$text($record['category'] ?? '')] ??
    $text($record['category'] ?? '')) ?></td>
72  <td data-label="Message" class="message"><?= nl2br(e($text($record['message'] ?? ''))) ?></td>
```

8. JSON File Storage with File Locking

append_submission() opens private/submissions.json in c+ mode (create if missing, do not truncate), takes an exclusive lock with flock(LOCK_EX), reads and decodes the list, adds the record with the next ID (SUB-0001, SUB-0002, …), truncates the file and writes the whole list back, then releases the lock. Because the read–modify–write happens under one lock, simultaneous submissions cannot overwrite each other. records.php reads with a shared lock (LOCK_SH).

- **Missing file or folder:** the folder is created and the file starts as an empty list.

- **Damaged file:** invalid JSON or JSON that is not a list is copied to submissions.corrupt-<date-time>.json before a new list is started, so nothing is lost silently; records.php shows a clear message instead of failing.
- **Write errors:** a failed open, lock or write throws an exception; the user sees “your inquiry could not be saved because of a server problem” (HTTP 500) and the details go to the PHP error log.
- **Stored fields:** id, submitted_at (ISO date-time, Indian time), name, email, category, message. No IP address or browser details are stored.

```
{ "id": "SUB-0001", "submitted_at": "2026-10-30T14:10:50+05:30", "name": "Eshan Kapoor",
  "email": "eshan.kapoor@example.com", "category": "portal",
  "message": "I cannot open the attendance page after logging in." }
```

8.1 storage.php – append_submission()

```
40  // Adds one record. The whole read-modify-write happens under an exclusive lock (flock LOCK_EX),
41  // so two submissions at the same time cannot overwrite each other.
42  // A missing file or folder is created; a damaged file is kept as a .corrupt copy and a new list is
    started.
43  function append_submission(array $record, ?string $file = null): array
44  {
45      $file ??= storage_file();
46      $dir = dirname($file);
47      if (!is_dir($dir) && !mkdir($dir, 0775, true) && !is_dir($dir)) {
48          throw new RuntimeException('The storage folder could not be created.');
49      }
50      $handle = @fopen($file, 'c+');
51      if ($handle === false) {
52          throw new RuntimeException('The storage file could not be opened.');
53      }
54      try {
55          if (!flock($handle, LOCK_EX)) {
56              throw new RuntimeException('The storage file could not be locked.');
57          }
58          $raw = (string) stream_get_contents($handle);
59          $records = decode_records($raw);
60          if ($records === null) {
61              $copy = $dir . '/submissions.corrupt-' . date('Ymd-His') . '.json';
62              if (file_put_contents($copy, $raw) === false) {
63                  throw new RuntimeException('The storage file is damaged and could not be backed up.');
64              }
65              $records = [];
66          }
67          $record = ['id' => next_id($records)] + $record;
68          $records[] = $record;
69          $json = json_encode($records, JSON_OPTIONS) . "\n";
70          if (!ftruncate($handle, 0) || !rewind($handle) || fwrite($handle, $json) !== strlen($json)) {
71              throw new RuntimeException('The storage file could not be written.');
72          }
73          fflush($handle);
74          return $record;
75      } finally {
76          flock($handle, LOCK_UN);
77          fclose($handle);
78      }
79  }
```

9. Success and Error Messages

- **Success:** after a redirect, “Inquiry submitted successfully – Thank you, <name>. Your inquiry has been saved with reference SUB-0005.” (role="status"). It is stored in the session and shown once.
- **Validation errors:** a summary “Please correct N fields” with links to the fields (role="alert"), a message under each invalid field, a red border, aria-invalid="true" and aria-describedby, and the entered values kept so the user only fixes the mistakes.

- **Other errors:** an expired or missing CSRF token, or a storage failure, shows a separate message explaining what to do.

9.1 index.php – error summary

```
 99  <?php elseif ($errors): ?>
100          <div class="alert alert-error" role="alert" id="error-summary">
101              <h2>Please correct <?= count($errors) ?> <?= count($errors) === 1 ? 'field' : 'fields' ?
     ></h2>
102              <ul>
103  <?php foreach ($errors as $field => $message): ?>
104                  <li><a href="#<?= e($field) ?>"><?= e($message) ?></a></li>
105  <?php endforeach; ?>
106              </ul>
107          </div>
108  <?php endif; ?>
```

10. Extensions, Security and Privacy

- **Display stored records (intermediate):** records.php shows a table of all submissions (newest first) and the JSON file contents, all escaped. It is labelled as a local demonstration page; a real site would restrict it to staff.
- **CSRF token (advanced):** csrf_token() creates a random 64-character token (random_bytes) in the session; csrf_is_valid() compares it with hash_equals(). A new token is created after each successful submission, so an old one cannot be reused.
- **Private storage:** private/.htaccess and includes/.htaccess deny access under Apache; router.php does the same for PHP's built-in server (including upper-case and backslash variants of the path).
- **Not committed:** Practical-7/.gitignore excludes private/*.json and private/*.csv, so real submissions never reach GitHub. sample-output/submissions.sample.json contains only the fictional records created during the screenshot run (example.com addresses).

10.1 functions.php – CSRF token

```
108  // CSRF protection: a random token kept in the session and sent back in a hidden field.
109  function csrf_token(): string
110  {
111      if (empty($_SESSION['csrf_token'])) {
112          $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
113      }
114      return $_SESSION['csrf_token'];
115  }
116
117  function csrf_is_valid(mixed $token): bool
118  {
119      return is_string($token) && isset($_SESSION['csrf_token']) && hash_equals($_SESSION['csrf_token'],
     $token);
120  }
```

11. Browser Screenshots

The screenshots are genuine renders of the running PHP application (XAMPP PHP 8.2.12 built-in server with router.php) in Microsoft Edge 156 (headless). A small page served by the same server loaded the real form in a frame, typed the values and submitted it with a normal POST. The screenshot run used its own storage file, so private/submissions.json was not touched; the records in it are fictional (Section 10). The empty margins on both sides of the 1280-pixel screenshots are not shown here; the full images are in Practical-7/screenshots/.

11.1 Form

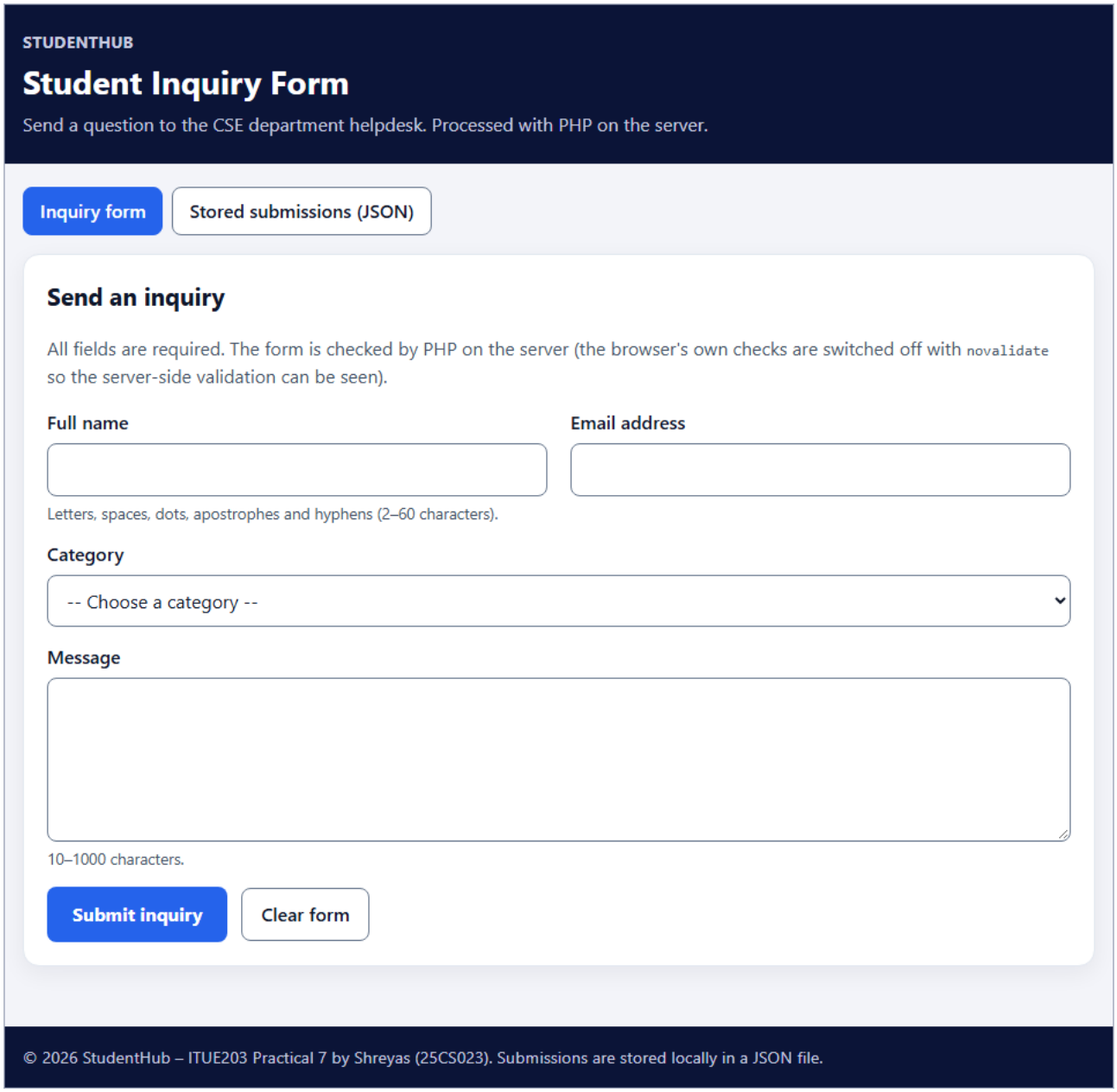


Figure 1: The inquiry form – four labelled fields, POST to index.php – form-ui.png

11.2 Validation errors

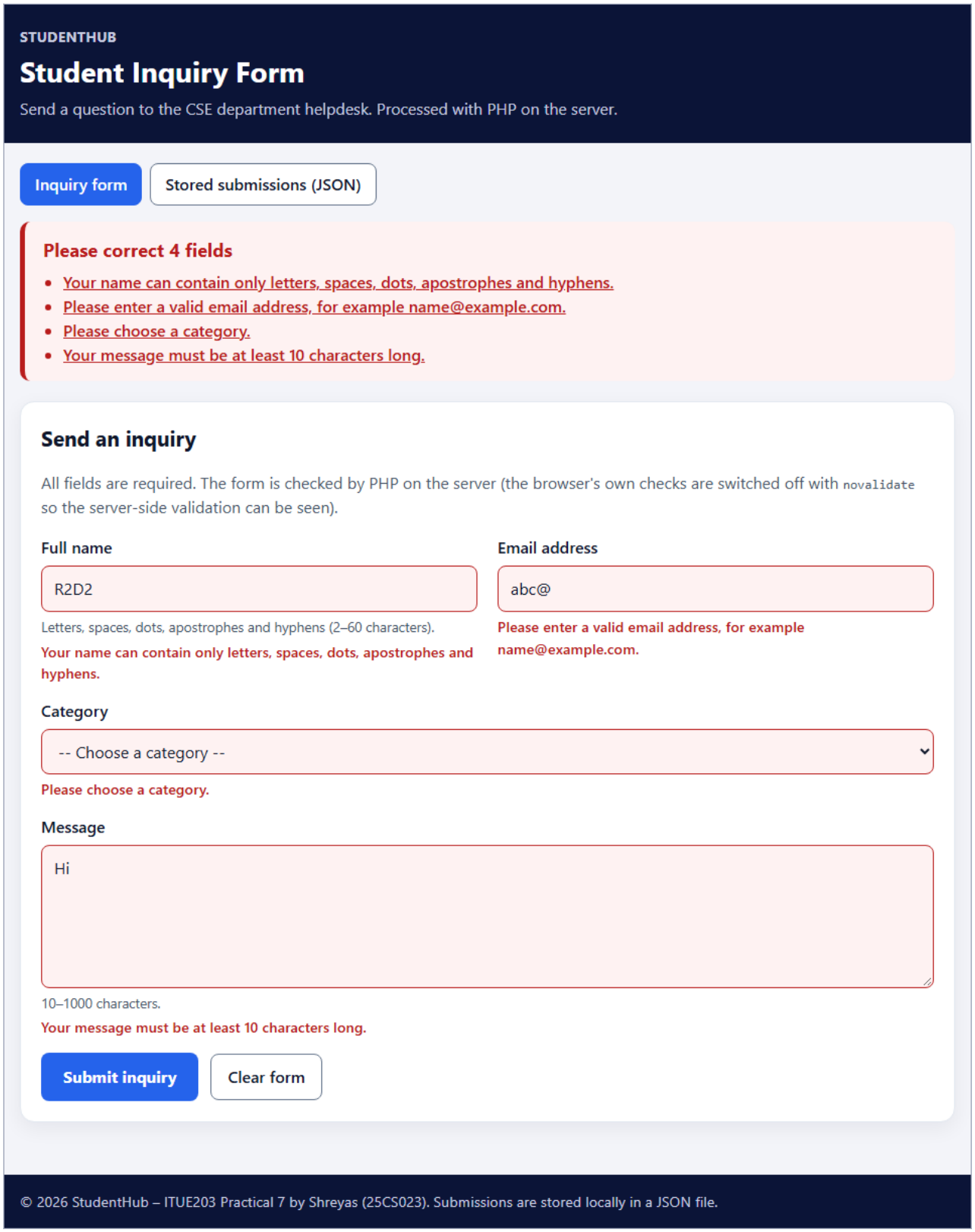


Figure 2: Server-side validation after submitting name "R2D2", email "abc@", no category and message "Hi" – summary of 4 fields and a message under each field, values kept – validation-error.png

11.3 Successful submission

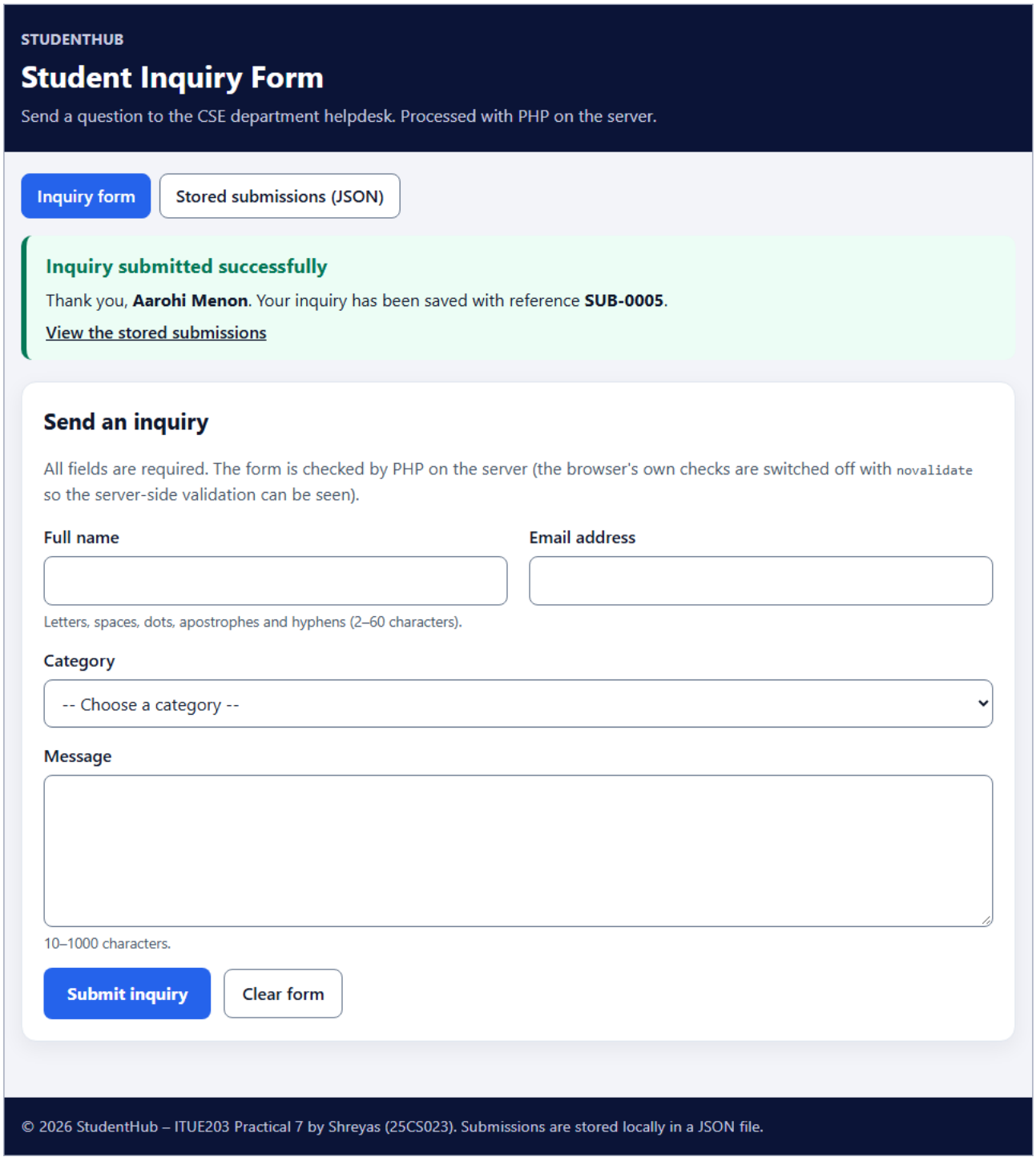


Figure 3: After a valid submission and the 303 redirect – "Inquiry submitted successfully" with reference SUB-0005 – successful-submission.png

11.4 Stored JSON data



Figure 4: records.php – 5 records read from the JSON file and the file contents; the <script> message (SUB-0003) is displayed as text – json-stored-data.png

12. Test Cases and Results

tests/test_form.py (Python standard library) starts XAMPP's PHP server with router.php and a separate temporary JSON file, then sends real HTTP requests like a browser: it keeps the session cookie, reads the CSRF token from the form and posts the fields. It checks status codes, page text and the JSON file. Result: 63 of 63 checks passed, and all five PHP files pass php -l.

#	Test case	Input	Expected result	Result
1	Form opens	GET index.php and /	200; POST form, 4 labelled fields, CSRF token	Pass
2	Empty submission	All fields empty	422; “Please correct 4 fields”; message per field; nothing stored	Pass
3	Invalid email	abc@, name@domain, a b@example.com, @example.com, 107-character address	422; email message only; other values kept	Pass
4	Invalid input	Name R2D2 / A / 61 chars; category hacker; message “Too short” / 1001 chars; name[] array; invalid UTF-8	422 with the matching message; nothing stored	Pass
5	Valid submission	Aarohi Menon, valid email, category, message	303 → success message with SUB-0001, shown once	Pass
6	JSON storage	After test 5	Valid pretty JSON list; 6 fields; sanitised name and email; ISO time	Pass
7	Multiple submissions	3 in sequence, then 10 at the same time	14 records, unique IDs SUB-0001–0014, valid JSON	Pass
8	Safe display	Name with apostrophe; message with <script>, <, & and quotes; HTML in a rejected name	Shown as escaped text; no <script> tag; JSON stores \u003C	Pass
9	CSRF	No token, wrong token, reused token	400 with a clear message; nothing stored	Pass
10	Storage problems	File deleted; invalid JSON; JSON object; missing folder	Handled; damaged file backed up; folder created	Pass
11	Private files	/private/…, /PRIVATE/…, /private%5C…, /includes/…, /tests/…, /.gitignore, /router.php	403 Forbidden	Pass
12	GET does not store	Form data in the address	Nothing stored	Pass
13	PHP errors	Server log of the whole run	No warnings, notices or errors	Pass

The launcher start-server.bat was also tested with its nobrowser option: it started XAMPP's PHP on port 8607 from another folder, served index.php, records.php and the stylesheet, and returned 403 for private/submissions.json.

12.1 Problems found and fixed

- **Test script:** the first run reported 10 failures caused by two mistakes in the test itself – a header name with different capitalisation, and an “array” test that actually sent a normal text value, which was then stored and shifted later counts. Both were corrected; the application code did not change.
- **records.php:** the JSON heading named private/submissions.json even when another storage file is configured (as in the screenshot run), and record IDs wrapped onto two lines. The heading now says “the JSON storage file” and the ID column no longer wraps.

12.2 Not tested

- XAMPP Apache (Option B) – only PHP's built-in server was used; the .htaccess rules were therefore not tested.
- Other browsers, screen readers, real phones, and a person using a real keyboard and mouse.
- A storage folder without write permission (the error path exists in the code but was not triggered).

13. Results and Conclusion

The StudentHub inquiry form is processed entirely by PHP. Data is sent with POST, sanitised and validated on the server, and valid records are appended to a JSON file under an exclusive file lock, with missing and damaged files handled safely. Users see a clear success message with a reference number, or a summary and field-level messages that keep their input. Every displayed value is escaped, so submitted HTML is shown as text.

Both extensions are included: a page that displays the stored JSON records (intermediate) and CSRF token validation (advanced). The automated tests – empty, invalid and valid submissions, JSON storage, multiple and simultaneous submissions, safe display, CSRF, damaged files and protected folders – passed 63 of 63 checks, and the four required screenshots show the form, validation errors, a successful submission and the stored data. Real submissions stay on the local computer and are never committed.

14. References

- ITUE203 Web Development Frameworks – Practical List 2026-27, Practical 7: “PHP Form Processing with Server-Side Validation and CSV/JSON File Storage”, CSPIT, CHARUSAT.
- PHP Manual – Dealing with forms: https://www.php.net/manual/en/tutorial.forms.php
- PHP Manual – filter_var and validate filters: https://www.php.net/manual/en/filter.filters.validate.php
- PHP Manual – htmlspecialchars: https://www.php.net/manual/en/function.htmlspecialchars.php
- PHP Manual – flock: https://www.php.net/manual/en/function.flock.php
- PHP Manual – json_encode: https://www.php.net/manual/en/function.json-encode.php
- OWASP Cheat Sheet – Cross-Site Request Forgery Prevention: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- StudentHub repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023